AI Agent 面试八股笔记

面向 Go 后端 / AI Agent 开发实习

一、大模型基础

二、Prompt 工程与推理增强

三、Agent 基础与设计范式

四、工具调用：Function Calling、MCP、Skill

五、RAG 检索增强生成

六、记忆与上下文工程

七、多 Agent 系统

八、评估、稳定性与可观测性

九、安全与人机协作

十、Agent Harness 与工程化

十一、开发框架

> 使用说明：按「主题 → 知识点」组织的结构化笔记，知识框架主要参考小林coding（小林面试笔记）AI 系列面试题：大模型工程、Agent、RAG、LLM 工具调用、LangChain，已重新整理改写，并补充少量论文与官方规范出处。AI 领域变化快，涉及具体版本/发布时间处已标注，面试时以官方最新文档为准。

一、大模型基础

1. 大语言模型与 Transformer

- **LLM**：基于 Transformer、在海量文本上**自监督预训练**（预测下一个 token）的超大参数模型；与传统 NLP 模型（每个任务单独训练一个小模型）相比，LLM 是**通用底座**，通过 Prompt / 少量示例 / 微调即可适配多种任务，具备上下文学习（in-context learning）与指令遵循能力。
- **Transformer**（2017《Attention Is All You Need》）：
  - **Encoder**：双向自注意力，擅长理解（BERT 系列）；
  - **Decoder**：带**因果掩码**的自注意力，只能看到前文，擅长生成（GPT 系列）；
  - 主流 LLM 为 **Decoder-only** 架构；T5 等为 Encoder-Decoder。
- **自注意力**：`Attention(Q, K, V) = softmax(QKᵀ / √d_k) · V`。除以 √d_k 是为了防止点积过大导致 softmax 进入饱和区、梯度过小。

- **多头注意力（MHA）**：多组 Q/K/V 并行计算，捕获不同子空间的关系后拼接。
- 每层结构：注意力 + 前馈网络（FFN），配合**残差连接**和 **LayerNorm**（现代模型多用 Pre-Norm、RMSNorm）。

2. 注意力优化：MQA、GQA、FlashAttention

- MHA 的局限：推理时每个头都要缓存 K/V，**KV Cache 显存占用大、访存带宽成为瓶颈**；注意力计算复杂度随序列长度 $O(n^2)$。
- **MQA（多查询注意力）**：所有头**共享一组 K/V**，KV Cache 大幅减小，但可能损失质量。
- **GQA（分组查询注意力）**：头分成若干组，**组内共享 K/V**，在 MHA 与 MQA 之间折中（Llama 2 70B、Llama 3 等采用）。
- **FlashAttention**：不改变计算结果的**精确注意力**，通过**分块（tiling）+ 在 SRAM 中融合计算**，减少对显存（HBM）的读写，提速并降低显存占用。

3. 位置编码与分词器

- 自注意力本身对顺序不敏感，需要**位置编码**注入位置信息。
  - **sin/cos 绝对位置编码**：原始 Transformer，不同频率三角函数与词向量相加；
  - **RoPE 旋转位置编码**：对 Q、K 按位置做旋转，使注意力分数只依赖**相对位置**，长度外推较好，现为主流（Llama、Qwen 等）；
  - **ALiBi**：不加位置向量，直接在注意力分数上加与距离成正比的**线性偏置**，外推能力强。
- **分词器（Tokenizer）**：把文本切成 token 并映射为 ID。主流为子词算法：
  - **BPE**：从字符/字节开始，反复合并最高频的相邻对（GPT 系列使用字节级 BPE）；
  - WordPiece（BERT）、Unigram / SentencePiece（T5、Llama 早期）。
  - 子词方案兼顾词表大小与未登录词问题；token 数决定**计费和上下文占用**，中文在不同 tokenizer 下的 token 效率差异较大。

4. 大模型训练流程与 Scaling Law

- **预训练**：海量无标注语料上做下一个 token 预测，学习语言与世界知识，算力消耗最大。
- **后训练（Post-Training）**：
  - **SFT 监督微调**：用高质量「指令-回答」数据教会模型遵循指令、对话格式；
  - **偏好对齐**：RLHF、DPO 等，让输出更符合人类偏好（有用、无害、诚实）；
  - 推理模型还会用**基于可验证奖励的强化学习**（数学、代码可自动判对错）提升推理能力。
- **Scaling Law**：模型损失随**参数量、数据量、计算量**按幂律平滑下降（Kaplan 等 2020）；Chinchilla（2022）指出在给定算力下应同时扩大参数和数据，经验上训练 token 数约为参数量的 20 倍较优。
- **涌现能力**：某些能力（多步推理、上下文学习）在规模达到一定阈值后才明显出现；也有研究认为这与评价指标的不连续性有关。

5. 微调方案与 LoRA

- **全参数微调**：效果上限高，但显存与存储成本大，且易**灾难性遗忘**。
- **参数高效微调（PEFT）**：Prompt Tuning、Prefix Tuning、Adapter、**LoRA / QLoRA**。
- **LoRA**：冻结原权重 W，只训练低秩分解矩阵 $\Delta W = B \cdot A$（秩 r 远小于原维度），前向为 $W \cdot x + B \cdot A \cdot x$。
  - 优点：可训练参数极少、显存低；**推理时可将 BA 合并回 W，不增加延迟**；一个底座可挂多个 LoRA 适配器按任务切换；原模型不被破坏，遗忘较少。
  - 关键超参：秩 r、缩放系数 alpha、作用的层（通常为注意力的 q/k/v/o 投影及 FFN）。
- **QLoRA**：底座以 **4-bit（NF4）量化**加载，再训练 LoRA，单卡即可微调较大模型。
- 选择：**知识注入优先 RAG，格式/风格/特定能力固化用微调**。

6. 偏好对齐：RLHF、PPO、DPO、GRPO

- **RLHF**（InstructGPT 流程）：SFT → 用人类排序数据训练**奖励模型 RM** → 用 **PPO** 强化学习优化策略模型，并加 KL 惩罚防止偏离参考模型太远。
- **PPO**：在线强化学习，需要策略模型、参考模型、奖励模型、价值（Critic）模型，训练复杂、显存开销大、不够稳定。
- **DPO**（2023）：直接用偏好对（chosen / rejected）构造分类式损失优化策略，**不需要显式奖励模型和强化学习采样**，训练简单稳定，属离线方法；缺点是依赖偏好数据质量与分布。
- **GRPO**（DeepSeekMath 2024 提出，DeepSeek-R1 使用）：对同一问题采样一组回答，用**组内平均奖励作为基线**计算相对优势，**省去价值模型**，适合可验证奖励的推理任务。
- **拒绝采样**：对一个问题生成多个回答，用奖励模型或规则筛选最优者作为新的 SFT 数据。

7. 解码策略与采样参数

- **贪心解码**：每步取概率最大 token，确定性强但易重复、单调。
- **Beam Search**：保留 k 条候选序列，适合翻译、摘要等有标准答案的任务；开放式对话中易生成平淡内容。
- **采样**：
  - **Temperature**：缩放 logits，**越低越确定**（接近贪心），越高越多样/随机；
  - **Top-K**：只在概率最高的 K 个 token 中采样；
  - **Top-P（核采样）**：在累计概率达到 P 的最小 token 集合中采样，候选数随分布自适应。
- 场景建议：代码、结构化抽取、工具调用、事实问答用**低温度**（$0_{0.3}$）；创意写作用较高温度（$_{0.7}1.0$）；通常调 temperature 或 top-p 其一即可。另有重复惩罚（repetition / frequency penalty）。

8. KV Cache 与 Prompt Caching

- 自回归生成时，每个新 token 都要与之前所有 token 做注意力。**KV Cache** 缓存已计算的 K、V，新 token 只需计算自己的 Q/K/V，避免重复计算。
- 推理两阶段：**Prefill**（并行处理整个输入，决定**首 token 延迟 TTFT**）与 **Decode**（逐 token 生成，受显存带宽限制）。
- 代价：KV Cache 随**序列长度 × 层数 × 头数**线性增长，是长上下文显存的主要开销 → MQA/GQA、KV 量化、PagedAttention（vLLM）优化。
- **Prompt Caching**：服务端缓存相同**前缀**的 KV，命中后降低延迟和费用。要求前缀**完全一致**，因此应把**系统提示、工具定义、固定文档放在前面**，变化内容放在后面。

9. 模型量化

- 将权重（及激活/KV）从 FP16/BF16 压缩到 **INT8 / INT4** 等低比特，降低显存、提升吞吐，代价是一定精度损失。
- **训练后量化（PTQ）** vs 量化感知训练（QAT）。
- 常见方案：
  - **INT8**：如 LLM.int8()，对离群值特殊处理，精度损失小；
  - **GPTQ**：逐层利用二阶信息（近似 Hessian）补偿量化误差的 INT4/INT3 权重量化；
  - **AWQ**：根据**激活值**识别重要权重通道，通过缩放保护它们，INT4 下精度保持好；
  - **GGUF**：llama.cpp 使用的格式，适合 CPU / 端侧。
- 选择：显存充足优先 BF16/INT8；资源紧张用 INT4（AWQ / GPTQ）；端侧用 GGUF。量化后需在业务评测集上验证。

10. MoE 混合专家模型

- 将 FFN 层替换为**多个专家网络 + 路由器（Router/Gate）**，每个 token 只激活 Top-K 个专家。
- 优点：**总参数量大（知识容量大），每个 token 实际计算量小**，训练和推理性价比高。例：DeepSeek-V3 总参数 671B、每 token 激活约 37B。
- 难点：**负载均衡**（避免少数专家过载，常用辅助损失或无辅助损失的偏置调节）、专家并行带来的通信开销、显存需容纳全部专家参数。

11. 推理部署方案

- **vLLM**：**PagedAttention**（像操作系统分页一样管理 KV Cache，减少碎片）+ **连续批处理（Continuous Batching）**，高吞吐，提供 OpenAI 兼容接口，生产常用。
- **SGLang**：RadixAttention 前缀缓存复用，适合多轮对话、Agent 等前缀共享多的场景，支持结构化输出加速。
- **TGI**（Hugging Face）：与 HF 生态集成好。

- **llama.cpp / Ollama**：C/C++ 实现，支持 CPU、Apple Silicon，GGUF 量化，适合本地和端侧。
- 选型看：吞吐/延迟需求、硬件、模型支持、量化需求、运维成本；关注指标 **TTFT、TPOT（每 token 时延）、吞吐（tokens/s）、并发数**。

12. 模型评测与选型

- 通用能力：MMLU（知识）、GSM8K / MATH（数学）、HumanEval / SWE-bench（代码）、C-Eval / CMMLU（中文）、指令遵循（IFEval）、Chatbot Arena（人类偏好 Elo）。
- 业务落地更重要的是**自建评测集**：准确率、格式正确率、工具调用正确率、幻觉率、延迟、成本。
- 选型维度：效果、上下文长度、工具调用能力、延迟与价格、是否可私有化部署、数据合规、中文能力。常见做法是**大小模型路由**：简单任务用小模型，复杂推理用强模型。

13. 长上下文与 Lost in the Middle

- **Lost in the Middle**（Liu 等 2023）：关键信息放在上下文**开头或结尾**时模型利用得最好，放在**中间**时效果明显下降（U 形曲线）。
- 原因：训练数据中关键信息分布偏向首尾、位置编码对远距离衰减、注意力被大量无关内容稀释。
- **窗口不是越大越好**：成本和延迟随长度增加；噪声多导致注意力分散；有效利用长度往往小于标称长度。
- 对策：只放**相关**内容（检索 + 重排）、重要信息放首尾、结构化分段并加标题、长文档先摘要或分块处理。

二、Prompt 工程与推理增强

14. Prompt 编写原则

- **角色与目标清晰**：说明身份、任务、受众与成功标准。
- **提供上下文与约束**：背景资料、输入数据、禁止事项、长度和语气要求。
- **结构化**：用分隔符（XML 标签、Markdown 标题、三引号）区分指令与数据，降低指令混淆和注入风险。
- **Few-shot 示例**：给出 1~5 个高质量、多样化示例，示例格式即期望输出格式。
- **指定输出格式**：JSON Schema / 结构化输出，便于程序解析；配合校验与重试。
- **任务拆解**：复杂任务拆成多步或多次调用（Prompt Chaining）。
- **迭代优化**：基于评测集做 A/B 对比，Prompt 也要**版本管理**。
- 正向表述（告诉模型做什么）通常比只说不要做什么更有效；系统提示里写清优先级和冲突处理规则。

15. CoT 思维链

- **CoT（Chain-of-Thought）**：让模型先输出中间推理步骤再给答案（Wei 等 2022）；**Zero-shot CoT** 只需加「让我们一步步思考」（Kojima 等 2022）。
- 为什么有效：把复杂问题分解为多步，每步只需较简单的推理；生成的中间步骤成为后续 token 的上下文，相当于给模型「更多计算步骤」。
- 扩展：**Self-Consistency**（多次采样不同推理路径，多数投票）、**ToT 思维树**（树状搜索多条推理路径并评估回溯）。
- 局限：增加 token 消耗与延迟；推理步骤可能看似合理但错误（不忠实）；对简单任务收益小；对小模型效果有限。推理模型（如 o 系列、DeepSeek-R1）已在训练中内化了长思维链。

16. 幻觉成因与缓解

- **幻觉**：生成看似合理但与事实或输入不符的内容。分为**事实性幻觉**（与世界知识不符）与**忠实性幻觉**（与给定上下文/指令不符）。
- 成因：
  - 训练数据含错误、过时信息，存在**知识截止**；
  - 训练目标是「生成概率高的下一个 token」而非「说真话」，**不知道时也倾向于给出答案**；
  - 解码随机性；长上下文中信息利用不充分。
- 缓解：
  - **RAG** 提供可靠依据，并要求**引用来源**；
  - Prompt 中允许「不知道」，并约束只依据给定资料回答；
  - 降低 temperature、结构化输出；
  - **事后校验**：自我检查、交叉验证、规则/工具校验（如计算交给代码执行）；
  - 关键场景加人工审核。

三、Agent 基础与设计范式

17. 什么是 Agent，与大模型的区别

- **LLM** 本身是「输入文本 → 输出文本」的函数，被动、单次、无状态，不能直接作用于外部世界。
- **Agent** 是以 LLM 为「大脑」、能**自主感知环境、规划决策、调用工具执行动作，并根据反馈迭代直至完成目标**的系统。
- 本质区别：LLM 负责「想」，Agent 在此基础上增加了「**循环 + 行动 + 状态**」——多步自主执行、与环境交互、维护记忆。

- 经典定义（Lilian Weng 2023）：**Agent = LLM + 规划（Planning）+ 记忆（Memory）+ 工具使用（Tool Use）**。

18. Agent 的核心组件

- **大模型（大脑）**：理解意图、推理、决策下一步。
- **规划**：任务分解、子目标设定、反思与修正。
- **记忆**：短期记忆（当前上下文/会话状态）+ 长期记忆（外部存储，跨会话）。
- **工具**：搜索、代码执行、数据库、API、文件系统等，扩展模型能力边界。
- **执行循环与运行时（Harness）**：组织「调用模型 → 执行工具 → 回填结果」的循环，处理状态、权限、预算和终止条件。
- 另有：感知（多模态输入）、安全护栏、可观测性。

19. Workflow、Agent 与 Tools

- **Tools**：被调用的具体能力（函数/API），本身不做决策。
- **Workflow（工作流）**：由**开发者预先编排**好执行路径的流程，LLM 和工具作为节点，路径确定、可预测、易调试。
- **Agent**：由 **LLM 动态决定**下一步做什么、调用什么工具、何时结束，灵活但不确定性更高、成本更高。
- Anthropic《Building effective agents》（2024.12）总结的常用 Workflow 模式：
  - **Prompt Chaining** 链式调用（上一步输出作为下一步输入，中间可加校验）；
  - **Routing** 路由（先分类再分派到专门流程）；
  - **Parallelization** 并行（分段并行 sectioning / 多次投票 voting）；
  - **Orchestrator-Workers** 编排者-执行者（动态拆分子任务）；
  - **Evaluator-Optimizer** 评估-优化循环（一个生成、一个评审反馈）。
- **选型原则**：能用简单方案就不用复杂方案；路径固定、要求稳定 → Workflow；开放式、步骤无法预知 → Agent；实践中常**Workflow 为骨架、局部节点 Agent 化**。

20. ReAct 推理模式

- **ReAct**（Yao 等 2022，Reasoning + Acting）：交替进行 **Thought（思考）→ Action（调用工具）→ Observation（观察结果）**，循环直到给出 Final Answer。
- 实现：
  1. System Prompt 描述可用工具和输出格式（现在通常直接使用模型原生的 Function Calling）；
  2. 调用模型，解析输出：是工具调用则执行，并将结果作为 Observation 追加到消息历史；
  3. 再次调用模型，直到模型给出最终回答，或达到**最大步数 / 超时 / token 预算**。
- 优点：推理与行动结合，**根据真实反馈动态调整**，过程可解释，能减少纯推理的幻觉。

- 缺点：逐步决策，**步骤多时延迟和成本高**；缺乏全局规划，易陷入局部循环、重复调用；上下文随步数增长。

21. ReAct、Plan-and-Execute、Reflection 对比

范式	核心思想	优点	缺点	适用场景
ReAct	边想边做，每步观察后决定下一步	灵活、适应动态环境	缺全局规划，步骤多时成本高、易绕圈	探索性任务、工具交互多、信息逐步获取
Plan-and-Execute	先由规划器生成完整计划，再由执行器逐步执行，必要时**重规划**	全局视角、步骤清晰；执行可用小模型，省成本；可并行	初始计划可能不准，环境变化时需重规划	步骤可预见的复杂长任务
Reflection / Reflexion	执行后**自我评估**，生成反思经验用于下一次尝试	提升质量与成功率，从错误中学习	额外调用增加延迟和成本；自评可能不可靠	代码生成、写作等有明确质量标准、可验证的任务

- 实践中常组合：**Plan-and-Execute 做全局规划 + ReAct 执行子任务 + Reflection 做结果校验**。

22. 任务拆分与规划能力

- **为什么拆分**：复杂任务一次完成易遗漏、上下文过长；拆分后每步目标清晰、可单独验证、可并行，失败时只需重试局部。
- 拆分方式：
  - **按步骤顺序**（先调研后撰写）；**按领域/职责**（检索、分析、写作）；**按数据分片**（多个文件并行处理）；
  - 产出结构化计划（步骤、依赖、预期产物、验收标准），可表示为 **DAG** 以并行执行无依赖步骤。
- **赋予 LLM 规划能力的方法**：
  - Prompt 层：CoT、任务分解提示、Few-shot 计划示例；
  - 架构层：Plan-and-Execute、ToT / 搜索、**外部规划器**（如 LLM+P 将问题转为 PDDL 交给经典规划器）；
  - 训练层：用规划/推理数据微调、强化学习训练推理模型；
  - **重规划机制**：根据执行反馈动态修正计划。
- 效果提升：每个子任务给清晰指令和上下文、设置检查点与验证、失败重试与回退、控制拆分粒度（过细会增加协调开销）。

23. 反思机制

- 目的：LLM 一次输出可能有错，通过**自我审视 + 外部反馈**发现并修正错误。
- **Reflexion**（Shinn 等 2023）：Actor 执行 → Evaluator 评估（单测结果、规则、LLM 打分）→ Self-Reflection 生成文字反思存入记忆 → 下一轮尝试参考反思。

- 实现方式：
  - **自我批评**：同一模型以审阅者角色检查并修改；
  - **生成-评审分离**：用不同 Prompt 或不同模型担任评审（Evaluator-Optimizer）；
  - **基于外部信号**：编译错误、单测结果、工具报错——**比纯自评更可靠**。
- 注意：设置最大反思轮数；没有外部信号时自评可能「自我确认」错误答案。

24. 手搓 Agent vs 使用框架

- 选择手写的原因：框架**抽象层多、黑盒**，调试和定制困难；版本迭代快、API 不稳定；核心循环本身并不复杂（几百行即可）；对**性能、延迟、可观测性和安全**有严格要求；需要与已有后端技术栈（如 Go 服务）深度集成。
- 选择框架的原因：快速原型、复用现成集成（模型、向量库、工具）、内置持久化与人机协作能力。
- 实践建议：理解原理后再用框架；生产中常「**核心循环自研 + 选择性复用组件**」。

四、工具调用：Function Calling、MCP、Skill

25. Function Calling 原理

- 定义：让模型以**结构化格式（JSON）**输出要调用的函数名和参数，由**应用程序执行**，再把结果回传给模型。**模型本身不执行函数**。
- 流程：
  1. 请求中传入工具列表（名称、描述、**JSON Schema 参数定义**）；
  2. 模型判断是否需要工具，若需要则返回 `tool_calls`（可一次返回多个，即**并行工具调用**）；
  3. 应用解析参数、校验并执行，将结果以 `tool` 角色消息追加到对话；
  4. 再次调用模型，模型基于结果继续调用工具或生成最终回答。
- 可通过 `tool_choice` 控制：自动、必须调用、指定某个工具或禁止调用。
- **模型如何学会调用工具**：在 SFT 阶段加入大量「工具描述 + 用户请求 → 结构化调用 → 工具结果 → 最终回答」的数据进行训练（Toolformer 2023 等探索了自监督方式），再通过偏好对齐/强化学习提升选择工具和填参数的准确率；推理时可配合**约束解码**保证输出符合 Schema。

26. 工具设计与容错

- **工具设计要点**：名称和描述清晰（写明何时用、何时不用）；参数尽量少、类型明确、给出枚举和示例；返回结果精简、对模型友好（过长需截断或分页）；**错误信息可读、可指导修正**；写操作保证**幂等**；遵循最小权限。
- **容错策略**：

- 格式非法 / 参数错误：**Schema 校验**，把具体错误信息回传给模型让其修正重试（限制次数）；
- 超时 / 失败：超时控制、**指数退避重试**（仅对幂等操作）、熔断与降级（备用工具或返回部分结果）；
- 工具结果区分「执行失败」「结果为空」「超时状态未知」，不要让模型基于含糊结果继续推理；
- 达到失败阈值后交给人工或明确告知用户。

27. Tool Routing：工具很多时怎么办

- 问题：工具定义全部塞进上下文会**消耗大量 token**，且工具越多越容易选错。
- 方案：
  - **工具检索**：对工具描述做 embedding，按用户请求语义检索 Top-K 相关工具后再交给模型；
  - **分层路由**：先由路由模型识别意图/领域，再加载该领域的工具子集；
  - **多 Agent 分工**：每个子 Agent 只持有自己领域的少量工具；
  - **按需加载**：如 Skill 的渐进式加载，只暴露简要描述，需要时再加载详细内容；
  - 合并相似工具、优化描述消除歧义，并用评测集统计工具选择准确率。

28. MCP 模型上下文协议

- **MCP（Model Context Protocol）**：Anthropic 于 **2024 年 11 月**开源的开放协议，标准化 AI 应用与外部工具、数据源之间的连接方式，被称为 AI 应用的「USB-C 接口」。
- 解决的问题：原本 M 个应用对接 N 个工具需要 **M×N** 套集成；MCP 统一协议后变为 **M+N**，工具一次实现、多个应用复用。
- **架构组成**：
  - **Host**：用户直接使用的 AI 应用（如 IDE、桌面客户端）；
  - **Client**：Host 内部与某个 Server 保持 **1:1 连接**的协议客户端；
  - **Server**：对外暴露能力的轻量服务。
- **Server 提供的三类原语**：**Tools**（可执行函数，由模型决定调用）、**Resources**（可读取的数据/上下文，如文件、数据库记录）、**Prompts**（预置的提示模板）。
- **Client 提供的能力**：**Sampling**（Server 请求 Host 侧模型生成内容）、**Roots**（告知 Server 可操作的范围）、**Elicitation**（Server 向用户请求补充信息，2025 年规范新增）。
- **通信方式**：基于 **JSON-RPC 2.0** 消息；传输层支持 **stdio**（本地子进程）和 **Streamable HTTP**（远程服务，2025-03-26 版规范起取代早期的 HTTP + SSE 传输；可用 SSE 流式返回）。远程 Server 的授权基于 **OAuth 2.1**。
- 生命周期：初始化握手（协商协议版本和能力）→ 发现工具/资源（`tools/list`）→ 调用（`tools/call`）→ 关闭。

29. MCP 与 Function Calling 的区别与选型

- **层次不同**：Function Calling 是**模型层**能力（模型如何表达「调用某函数」）；MCP 是**应用/集成层**协议（工具如何被发现、描述、调用和传输）。
- 关系：MCP Client 拿到 Server 的工具列表后，**转换为 Function Calling 格式**交给模型；模型输出调用后，Client 再通过 MCP 转发给 Server 执行。二者**互补而非替代**。
- 选型：
  - 工具少、仅本应用内使用、追求低延迟 → 直接 Function Calling；
  - 工具需要**跨应用复用**、由第三方提供、希望生态即插即用 → MCP。
- 部分模型「不支持 MCP」的本质：模型没有经过工具调用训练或不具备 Function Calling 接口（如某些仅输出文本的推理模型版本），无法可靠产出结构化调用。

30. Agent Skill

- **Skill**：把完成某类任务所需的**指令、脚本、模板、参考资料打包成文件夹**的可复用能力包，核心文件为 `SKILL.md`（YAML frontmatter 声明 name、description + Markdown 正文写步骤与规范）。Anthropic 于 2025 年 10 月推出 Agent Skills，同年 12 月将规范作为开放标准发布。
- **渐进式加载（Progressive Disclosure）**：
  1. 启动时只加载所有 Skill 的**名称和描述**（很少 token）；
  2. 判断与任务相关时才读取 `SKILL.md` 正文；
  3. 正文引用的参考文档、脚本**按需读取或执行**。
  - 好处：可挂载大量 Skill 而不撑爆上下文。
- **与 MCP 的区别**：MCP 提供「**能访问什么**」（外部工具与数据的连接）；Skill 提供「**该怎么做**」（流程知识与最佳实践），Skill 内部可以调用多个 MCP 工具或脚本。
- **三者层级**：Function Calling（调用语言，最底层）→ MCP（工具标准化接入）→ Skill（任务流程封装，最上层）。

31. CLI 工具化与代码执行沙箱

- **把工具封装成 CLI 的理由**：模型在训练中见过大量命令行用法，调用自然；CLI 可**组合**（管道、脚本），输出可重定向到文件、按需读取，节省 token；本地开发与调试简单，无需常驻服务。
- **CLI vs MCP**：本地开发、模型已熟悉的命令、需要组合脚本 → CLI；需要统一鉴权、远程服务、跨客户端复用和结构化发现 → MCP。
- **代码执行沙箱设计**：
  - 隔离：容器 / gVisor / Firecracker 微虚拟机，独立文件系统；
  - 资源限制：CPU、内存、磁盘、**执行超时**、进程数；
  - **网络出口控制**（默认禁止或白名单）；

- 不注入敏感凭证，最小权限运行（非 root）；
- 审计日志，执行结果截断后返回模型；
- 会话结束销毁环境或按需持久化。

32. A2A 协议与实时通信方式

- **A2A（Agent2Agent）**：Google 于 2025 年 4 月发布的开放协议（后捐赠给 Linux 基金会），用于**不同厂商、不同框架的 Agent 之间**互相发现与协作。
  - **Agent Card**：JSON 格式描述 Agent 的能力、地址、认证方式，用于发现；
  - 以 **Task** 为核心，有明确的生命周期状态，支持长时间任务、流式更新（SSE）和推送通知；基于 HTTP + JSON-RPC。
  - **与 MCP 的区别**：MCP 解决 **Agent ↔ 工具/数据**（纵向）；A2A 解决 **Agent ↔ Agent**（横向）。二者互补。
- **SSE vs WebSocket**：
  - **SSE**：基于 HTTP 的**服务器 → 客户端单向**文本流，实现简单、自动重连、穿透代理友好，**LLM 流式输出最常用**；局限是单向、只能传文本、HTTP/1.1 下浏览器同域连接数有限。
  - **WebSocket**：**全双工**长连接，适合双向实时交互（协同、实时打断）；需要自行处理心跳、重连，部分网关/负载均衡配置更复杂。
- **WebRTC**：基于 UDP 的实时音视频传输，**低延迟**、内置回声消除/抖动缓冲，适合**实时语音 Agent**；WebSocket 基于 TCP，丢包时队头阻塞导致语音延迟和卡顿。

33. 大模型网关

- 位于应用与多个模型提供商之间的统一接入层，解决：
  - **统一 API**（屏蔽不同厂商接口差异，通常兼容 OpenAI 格式）；
  - **路由与容灾**：按成本/延迟/能力路由，主模型失败自动 **fallback**；
  - **限流与配额**、API Key 集中管理与租户隔离；
  - **缓存**（精确缓存、语义缓存）；
  - **可观测性**：token 用量、成本、延迟统计；
  - 内容安全过滤、审计合规。
- 常见实现：LiteLLM、Portkey、One API 等开源方案，或基于 API 网关（如 Higress）扩展。

五、RAG 检索增强生成

34. RAG 是什么、解决什么问题

- **RAG（Retrieval-Augmented Generation）**：生成前先从外部知识库**检索相关内容**，拼接到 Prompt 中，让模型**基于检索到的依据回答**。

- 解决的问题：
  - **知识时效性**：模型有知识截止日期，RAG 可接入最新数据；
  - **私有/领域知识**：企业内部文档无需训练进模型；
  - **幻觉**：提供依据并可**引用溯源**；
  - **权限控制**：检索时按用户权限过滤文档；
  - 成本：更新知识只需更新索引，无需重新训练。

35. RAG 与微调的对比

维度	RAG	微调
知识更新	实时，更新索引即可	需重新训练
可解释性	可引用来源	黑盒
幻觉控制	有依据，较好	仍可能幻觉
擅长	**注入事实性知识**、私有数据	改变**行为、风格、格式**，学习特定任务技能
成本	检索带来额外延迟和上下文 token	训练成本高，推理无额外检索开销
依赖	检索质量决定上限	训练数据质量

- 二者可结合：微调提升模型对领域术语的理解和「依据资料作答」的能力，RAG 提供实时知识。

36. 完整 RAG 工作流程

- **离线索引**：
  1. **文档加载**：PDF、Word、网页、数据库等，解析表格、图片（OCR）；
  2. **清洗**：去除页眉页脚、乱码、重复内容；
  3. **切分（Chunking）**；
  4. **Embedding** 向量化，同时保存**元数据**（来源、标题、时间、权限）；
  5. **写入向量数据库**（可同时建立关键词倒排索引）。
- **在线查询**：
  1. **Query 理解与改写**（补全指代、拆分子问题）；
  2. **多路召回**（向量检索 + 关键词检索），按元数据过滤；
  3. **融合与重排**（Rerank）；
  4. **构造 Prompt**：指令 + 检索片段（带来源编号）+ 用户问题；
  5. **LLM 生成**答案并**标注引用**；
  6. 后处理：引用校验、敏感信息过滤，记录日志用于评估。

37. 文档切分策略

- **固定长度切分**：按字符/token 数切分，设置**重叠（overlap，常见 10%~20%）**避免语义被截断。简单但可能切断句子。
- **递归字符切分**：按段落 → 句子 → 字符等分隔符层级递归切分，尽量保持语义完整（LangChain `RecursiveCharacterTextSplitter`）。
- **按文档结构切分**：按 Markdown 标题、HTML 标签、章节、代码函数切分，并将**标题路径**作为元数据或拼接到块开头。
- **语义切分**：计算相邻句子 embedding 相似度，在语义变化大的位置切分。
- **粒度权衡**：块太小语义不完整、召回碎片化；块太大噪声多、稀释相关性、占用上下文。常见起点为 **256~1024 token**，需根据文档类型和评测结果调整。
- **规避语义被切断的方法**：
  - 重叠窗口；按结构切分；
  - **父子块（Small-to-Big）**：用小块检索（精准），返回其所在的大块/父文档（完整）；
  - **句子窗口检索**：命中句子后扩展前后若干句；
  - 为每个块补充上下文摘要（如文档标题、所属章节说明）。

38. Embedding 原理、算法与选型

- **Embedding**：把文本映射为**稠密向量**，语义相近的文本向量距离近。相似度常用**余弦相似度**、内积、欧氏距离；向量归一化后余弦相似度等于内积。
- 算法演进：
  - 静态词向量：Word2Vec、GloVe（一词一向量，无法处理多义词）；
  - 上下文向量：BERT 类编码器，**Sentence-BERT** 用孪生网络 + 对比学习得到句向量；
  - 现代 Embedding 模型：基于对比学习大规模训练，如 **BGE、GTE、E5、OpenAI text-embedding-3、Jina** 等；
  - 表示形式：**稠密向量**（语义）、**稀疏向量**（BM25、SPLADE，关键词精确匹配）、**多向量**（ColBERT 延迟交互，token 级匹配）；BGE-M3 同时支持三种。
- **检索架构**：**双塔 Bi-Encoder**（查询和文档分别编码，文档可离线预计算，速度快，用于召回）vs **Cross-Encoder**（查询和文档拼接后一起编码，精度高但慢，用于重排）。
- **选型与评估**：参考 **MTEB / C-MTEB** 榜单；关注语言（中文）、领域适配、**向量维度**（影响存储和速度）、最大输入长度、是否可私有部署、成本；最终以**业务评测集上的 Recall@K、MRR** 为准。

39. 向量数据库与 ANN 索引

- **向量数据库**：存储向量及元数据，支持**近似最近邻（ANN）检索**、元数据过滤、增删改、水平扩展。
- 主要索引：

  - **Flat 暴力检索**：精确但 O(N)，适合小数据量；
  - **HNSW**：分层可导航小世界**图索引**，查询快、召回高，但内存占用大；关键参数 M（每节点连接数）、efConstruction、efSearch；
  - **IVF**：先 k-means 聚类成 nlist 个簇，查询时只搜最近的 nprobe 个簇；
  - **PQ 乘积量化**：将向量切段分别量化压缩，大幅节省内存，精度有损；常与 IVF 组合（IVF-PQ）；
  - DiskANN：基于磁盘的图索引，适合超大规模。
- 常见产品：**Milvus**（分布式、大规模）、**Qdrant**（Rust，过滤能力强）、Weaviate、Chroma（轻量、原型）、**pgvector**（PostgreSQL 扩展，与业务库统一）、**Elasticsearch / OpenSearch**（天然支持混合检索）、Redis；**Faiss** 是检索库而非数据库。
- 选型维度：数据规模、QPS 与延迟、是否需要过滤与混合检索、部署运维成本、是否已有技术栈。
- 性能瓶颈与优化：内存不足（量化、DiskANN）、过滤导致召回下降（预过滤/分区）、写入时索引构建慢（批量写入、异步建索引）、调节 ef / nprobe 平衡召回与延迟。

40. 向量检索与关键词检索、多路召回

- **向量（语义）检索**：理解同义、改写、跨语言；但对**专有名词、编号、代码、精确术语**不敏感。
- **关键词检索（BM25）**：基于词频和逆文档频率，精确匹配强、可解释、无需训练；但无法理解语义（同义词召回差）。
- **多路召回（混合检索）**：同时使用向量检索、BM25、元数据/结构化查询、知识图谱等多路召回，再**融合**：
  - **RRF（倒数排序融合）**：`score = Σ 1 / (k + rank_i)`，k 常取 60，只依赖排名，无需分数归一化；
  - 或加权分数融合（需归一化）。
- 融合后进入重排，取 Top-K 送入模型。

41. Query 改写与检索优化策略

- **Query 改写目的**：用户问题口语化、有指代、信息不完整，直接检索效果差。
  - **多轮对话补全**：结合历史把「它多少钱」改写为完整问题；
  - **多查询（Multi-Query）**：生成多个不同表述分别检索后合并；
  - **查询分解**：复杂问题拆成多个子问题分别检索；
  - **HyDE**（Gao 等 2022）：先让模型生成一个**假设性答案**，用它的向量去检索，缩小问答之间的语义差距；
  - **Step-back**：先抽象出更通用的问题检索背景知识。
- **检索优化全景**：

    - 索引侧：切分优化、元数据增强、为块生成摘要/假设问题建索引；
    - 检索侧：混合检索、元数据过滤、调整 Top-K；
    - 后处理：**Rerank 重排**、去重、上下文压缩（只保留相关句子）、相关性阈值过滤；
    - 生成侧：引用标注、要求依据不足时拒答。
- **Rerank**：用 **Cross-Encoder**（如 bge-reranker、Cohere Rerank）对召回的 Top-50~100 精细打分，取 Top-3~10；是提升 RAG 准确率最有效的手段之一。

42. 高级 RAG 范式与 GraphRAG

- 演进：**Naive RAG**（检索-生成）→ **Advanced RAG**（检索前优化 + 检索后重排压缩）→ **Modular RAG**（模块可组合编排）。
- 代表方法：
    - **Self-RAG**：模型自行判断是否需要检索，并对检索结果和生成内容进行反思评价；
    - **CRAG（Corrective RAG）**：评估检索质量，质量差时改写查询或转向网络搜索；
    - **Agentic RAG**：由 Agent 决定检索什么、何时检索、检索几次，支持多跳推理；
    - **RAPTOR**：对文档递归聚类摘要形成树状索引，兼顾细节与全局；
    - **GraphRAG**（微软 2024）：用 LLM 抽取实体与关系构建知识图谱，并做社区检测与摘要。
- **何时用图数据库增强**：问题涉及**多跳关系**（A 的供应商的竞争对手）、实体关系密集（金融风控、医疗、组织架构）、需要**全局总结类问题**（「这批文档的主要主题」）；代价是构建成本高、需要维护图谱质量。

43. RAG 幻觉控制与效果评估

- **规避幻觉**：提高检索质量（混合检索 + 重排）；Prompt 要求**仅依据资料回答、资料不足时明确说明**；要求逐句标注引用并做**引用校验**；设置相关性阈值，低于阈值不强行作答；生成后用 NLI 模型/LLM 检查答案是否被上下文支持。
- **评估指标**：
    - **检索层**：Recall@K、Precision@K、命中率 Hit Rate、**MRR**（第一个相关结果排名倒数的均值）、**NDCG**（考虑排序位置的增益）；
    - **生成层**（RAGAS 等框架）：**Faithfulness 忠实度**（答案是否被上下文支持）、**Answer Relevancy 答案相关性**、**Context Precision / Context Recall 上下文精确率/召回率**；
    - **端到端**：答案正确率、用户满意度、拒答率、延迟与成本。
- 方法：构建「问题-标准答案-相关文档」**黄金评测集**；结合 LLM-as-Judge 与人工抽检；每次改动做回归评测。

44. 知识库更新、知识冲突与落地难点

- **动态更新**：基于文档哈希/版本号做**增量索引**（只处理新增和变更的文档）；删除或更新时**同步删除旧 chunk**（按文档 ID 关联）；数据源变更通过 CDC/消息队列触发；记录时间戳元数据，

必要时按时间加权。

- **知识冲突**：按来源**权威性**（官方文档 > 论坛）和**时效性**（新版本优先）排序；元数据中保留版本与日期；无法判断时向用户**呈现不同说法并注明来源**，而不是自行编造调和。
- **落地难点**：复杂文档解析（表格、图片、扫描件）；切分策略难以通用；多跳和汇总类问题检索困难；评测集构建成本高；权限控制与数据安全；知识持续维护。

45. 长上下文时代为什么仍需要 RAG

- **成本与延迟**：每次把全部资料放入上下文，token 费用和 TTFT 高；RAG 只送相关片段。
- **规模**：企业知识库通常远超任何上下文窗口。
- **效果**：长上下文存在 Lost in the Middle、注意力稀释问题；精选内容往往更准确。
- **时效与权限**：RAG 易于更新，可按用户权限过滤检索结果。
- **可溯源**：可精确引用来源片段。
- 两者互补：长上下文让 RAG 可以召回更多、更完整的上下文。

46. RAG 与 Agent 自主文件检索的选型

- **Agent 自主检索**（像编程 Agent 用 grep / glob / 读文件逐步探索）：适合**结构化、可精确匹配**的语料（代码库、配置、日志），无需预建索引、实时准确，但多轮调用延迟和 token 成本较高。
- **RAG 向量检索**：适合**大规模非结构化**文档的语义检索，响应快，但需维护索引，精确匹配弱。
- 实践中常结合：RAG 快速定位候选文档，Agent 再深入读取与验证。

六、记忆与上下文工程

47. Agent 记忆机制

- **短期记忆（工作记忆）**：当前会话/任务的上下文，即每次调用模型时传入的 messages（用户输入、模型输出、工具结果、中间状态），受上下文窗口限制，任务结束可清理。
  - 进阶：**结构化工作记忆**——把当前目标、已确认结论、待办事项等放在固定区域，而不是无序堆叠消息。
- **长期记忆**：跨会话持久化，存于外部存储，按需检索后注入上下文。按内容可分为：
  - **情景记忆**（Episodic）：过去发生的交互和事件；
  - **语义记忆**（Semantic）：用户偏好、事实知识（如「用户偏好 Go 语言」）；
  - **程序性记忆**（Procedural）：做事的方法、规则、经验（可沉淀为系统提示或 Skill）。
- **记忆模块设计**：**写入**（由 LLM 判断哪些信息值得记、抽取为结构化记忆）→ **存储**（向量库 + KV/关系库 + 可选知识图谱）→ **检索**（语义相似度 + 时间近因 + 重要性综合打分，参考

Generative Agents 2023）→ **更新与遗忘**（新信息与旧记忆冲突时合并/覆盖，过期记忆衰减删除）。

- **存储粒度**：以「**一条完整、自包含的事实或一次关键事件**」为单位，太细检索碎片化，太粗噪声大；附带用户 ID、时间、来源、重要性等元数据，并做多租户隔离。

48. 记忆压缩方法

- **滑动窗口**：只保留最近 N 轮，简单但会丢失早期关键信息。
- **摘要压缩**：将早期对话用 LLM 总结为摘要，保留最近几轮原文（摘要 + 窗口混合最常用）。
- **重要性筛选**：保留关键决策、用户约束、未完成事项，丢弃寒暄和冗余工具输出。
- **工具结果裁剪/外置**：长日志、长文件只保留关键部分，完整内容存外部，留下引用路径以便后续按需读取。
- **分层记忆**：重要信息沉淀到长期记忆，需要时再检索回来。
- 注意：摘要会丢失细节且可能引入错误，关键约束（如用户明确要求）应原文保留。

49. 上下文压缩阈值的确定

- 不宜等到窗口完全占满才压缩：模型在接近窗口上限时效果下降，且需为后续输出与工具结果预留空间。
- 考虑因素：模型**有效上下文长度**、单轮最大输出与工具结果大小、成本预算、任务对历史细节的依赖程度。
- 常见做法：设置在窗口的一定比例（例如 70%~85%）触发，或按「剩余空间 < 预计下一轮所需空间」动态触发；**阈值应通过评测确定**（对比不同阈值下的任务成功率与成本）。

50. 上下文工程

- **上下文工程（Context Engineering）**：设计「**每一轮模型调用时，上下文里放什么、按什么结构放、放多少**」，比单纯写 Prompt 范围更广。
- 组成：系统指令、工具定义、长期记忆检索结果、RAG 资料、对话历史、当前任务状态、少量示例。
- 原则：
  - **相关性优先**：只放当前步骤需要的信息，避免无关内容干扰；
  - **结构稳定**：固定内容放前面，利用 Prompt Caching；
  - **关键信息置于首尾**，用标签分隔不同来源；
  - **按需加载**：大文件和工具结果外置，提供检索/读取工具；
  - 子 Agent 隔离上下文，主 Agent 只接收其摘要结果。

51. 多轮对话状态管理与中断恢复

- **状态与上下文分离**：任务状态（当前步骤、已完成操作、产物位置、待确认事项）持久化在**外部存储**；每轮上下文从状态中选取需要的部分。
- **防止跑偏**：每轮注入明确的**任务目标与约束**；维护计划与进度清单（todo）；定期检查当前行动是否服务于目标；偏离时重规划或请求用户澄清。
- **中断恢复**：
  - 关键步骤后写入**检查点（Checkpoint）**，如 LangGraph 的 checkpointer 按线程保存图状态；
  - 恢复时读取检查点，并**核对外部实际状态**（文件是否已写入、请求是否已执行成功），不能仅凭对话记录判断；
  - 写操作设计为**幂等**，避免重复执行造成副作用。

七、多 Agent 系统

52. Multi-Agent 概念与适用场景

- **Multi-Agent**：多个各有角色、工具和上下文的 Agent 协作完成任务。
- Single-Agent 足够的场景：流程清晰、上下文可容纳、不需要明显分工 —— **优先 Single-Agent**，实现简单、好调试。
- 引入 Multi-Agent 的典型理由：
  - **上下文过长**：单个 Agent 装不下，需拆分并隔离上下文；
  - **专业分工**：不同步骤需要差异很大的提示、工具和能力；
  - **可并行**：多个独立子任务并行加速（如深度调研同时搜索多个方向）。
- 代价：token 成本成倍增加、协调和通信开销、错误在 Agent 间传播、调试困难。

53. Multi-Agent 架构与协作机制

- **中心化（Orchestrator / Supervisor）**：主 Agent 负责理解目标、拆分任务、分派给 Worker、汇总结果；Worker 上下文干净、只做自己的部分。可控、链路清晰、易调试，**工程中最常用**。变体：静态路由 → 动态规划 → 自适应编排。
- **层级式**：多层 Supervisor，适合超大规模任务。
- **去中心化（Peer-to-Peer / Handoff）**：Agent 之间直接通信或移交控制权（如 OpenAI Agents SDK 的 handoff）；灵活，但**难以控制全局状态、易出现循环对话、难以追踪和终止**，工程上较少完全去中心化。
- **协作与动态切换**：
  - 明确每个 Agent 的**职责边界与输入输出契约**（结构化数据）；
  - 共享状态（黑板 / 全局 State）或消息传递；

    - **路由/移交**：由 Supervisor 或路由模型根据意图切换 Agent，切换时传递必要上下文摘要；
    - 设置最大轮次与终止条件，防止 Agent 之间无限对话。
- 常见框架：LangGraph（Supervisor、Swarm）、AutoGen、CrewAI（角色 + 任务）、MetaGPT（软件公司 SOP）、OpenAI Agents SDK。

54. Multi-Agent 的容错与一致性

- **子 Agent 超时 / 失联**：为每个子任务设置**超时**和心跳；失败重试（有限次数）或切换备用 Agent；允许**部分结果降级返回**并明确标注缺失部分；记录任务状态以便重新调度。
- **并发修改冲突**：
    - 划分资源所有权，**不同 Agent 写不同资源**；
    - 对共享资源加锁或使用**乐观并发控制（版本号）**；
    - 由 Orchestrator 统一合并结果，冲突时按规则或交给人工裁决。
- **错误传播**：每个环节输出做校验，不让错误结果无检查地流入下游。

八、评估、稳定性与可观测性

55. Agent 效果评估

- **评测集设计**：覆盖典型任务、边界情况、失败案例与对抗样例；记录标准答案或**可验证的成功条件**（如单测通过、数据库状态正确）；从线上真实日志中持续补充。
- **指标**：
    - **任务成功率**（端到端是否达成目标）；
    - **过程指标**：工具选择准确率、参数正确率、平均步数、重复/无效调用率；
    - **轨迹评估**：执行路径是否合理、是否有不必要步骤；
    - **效率与成本**：延迟、token 消耗、费用；
    - **安全**：越权操作次数、需人工介入比例。
- **方法**：规则/程序化校验（最可靠）、**LLM-as-Judge**（注意位置偏差、冗长偏差、自我偏好偏差，需与人工评估对齐校准）、人工评估；同一用例**多次运行**统计稳定性（τ-bench 提出 pass^k 衡量 k 次全部成功的一致性）。
- 公开基准：SWE-bench（代码修复）、GAIA（通用助手）、WebArena（网页操作）、τ-bench（工具 + 用户交互）、BFCL（函数调用排行榜）、AgentBench。

56. 路径震荡、重复调用与死循环

- **原因**：工具返回的错误信息不明确，模型反复尝试同样操作；缺少进度感知、忘记已完成步骤（上下文被截断或压缩）；目标不清或无法完成却没有退出机制；多个 Agent 互相推诿。

- **检测**：记录调用历史，检测**相同工具 + 相同参数**的重复调用；状态无变化的连续轮次；步数、token、时间超预算。
- **治理**：
  - 硬限制：**最大步数、超时、预算**；
  - 工具返回**明确、可行动的错误信息**；
  - 维护**任务清单与进度**，让模型知道已做什么；
  - 检测到重复时注入提醒或触发**反思/重规划**；
  - 超过阈值时**人工介入**或优雅失败并说明原因。

57. 延迟定位与 Trace

- **Trace**：为每次请求记录完整调用链，每次 LLM 调用、工具调用、检索都是一个 **Span**，记录耗时、输入输出、token 数、模型名、错误；可基于 **OpenTelemetry**，或使用 LangSmith、Langfuse 等平台。
- **定位思路**：查看延迟瓶颈在哪类 Span —— LLM 的 TTFT/生成时间、工具调用耗时、检索耗时、排队时间；对比正常与异常请求的步数和 token 数。
- **优化**：
  - **无依赖的工具调用并行执行**；
  - 简单步骤路由到**更小更快的模型**；
  - 减少上下文长度、利用 **Prompt Caching**；
  - 缓存工具结果与检索结果；
  - **流式输出**降低感知延迟；
  - 减少不必要的推理步骤与反思轮次。

58. 任务幻觉

- **任务幻觉**：Agent 声称「已完成」「已修改文件」「已发送邮件」，但实际**没有调用工具或工具执行失败**。
- 原因：模型倾向生成「完成态」话术；工具失败信息不清晰；上下文压缩后丢失执行记录。
- 避免：
  - **完成判断由程序而不是模型说了算**：以工具真实返回结果和外部状态校验为准；
  - 要求最终报告**引用具体执行证据**（工具调用 ID、文件路径、命令输出）；
  - 增加**验证步骤**（读回文件、运行测试、查询数据库确认）；
  - 区分「计划要做」与「已经做完」，在 UI 中展示真实执行记录。

59. 执行结果不一致的原因

- 采样随机性（temperature、top-p）；

- 推理服务端的非确定性：批处理组合不同、浮点运算顺序差异，即使 temperature 为 0 也可能结果不同；
- 模型版本更新；工具和外部环境返回结果变化（搜索结果、时间）；
- 早期步骤的微小差异在多步执行中**被放大**（路径依赖）。
- 缓解：低温度、固定模型版本和（若支持）随机种子；结构化输出约束；关键步骤用 Workflow 固化；评测时多次运行看分布，而不是只看一次结果。

九、安全与人机协作

60. Prompt 注入与 Agent 安全

- **直接注入**：用户输入「忽略之前的指令……」试图覆盖系统设定（越狱）。
- **间接注入**：恶意指令藏在 Agent 读取的**网页、文档、邮件、工具返回结果**中，诱导 Agent 执行非预期操作（如泄露数据、调用危险工具）。OWASP LLM Top 10 将 Prompt Injection 列为首位风险。
- 防御（纵深防御，没有单一银弹）：
  - **区分可信指令与不可信数据**：外部内容用分隔标签包裹，并声明其中的指令不得执行；
  - **最小权限**：工具权限按需授予，读写分离，敏感操作需二次确认；
  - **人在回路**：高风险操作（转账、删除、对外发送）必须人工确认；
  - 输入输出过滤与检测模型（护栏 Guardrails）；
  - 敏感信息不放进 Prompt，输出做脱敏；
  - 沙箱执行代码、限制网络出口；
  - 审计日志与异常告警。
- MCP 场景特别注意：**工具投毒**（工具描述中隐藏恶意指令）、来源不明的 Server、过度授权。

61. Agent 连接数据库的安全

- **防越权**：
  - Agent 使用**专用只读账号**，最小权限；按用户身份做**行级/列级权限**过滤；
  - 不让模型自由拼接任意 SQL：优先提供**参数化的预定义查询工具**；若开放 Text-to-SQL，需经 SQL 解析校验（只允许 SELECT、限制表白名单、强制 LIMIT、超时）。
- **防敏感数据泄漏**：敏感字段脱敏或不暴露给模型；输出检测敏感信息；记录审计日志。
- **防查询幻觉**：提供准确的表结构与字段说明（含枚举含义）；先生成 SQL 再执行并基于**真实结果**回答；SQL 报错时把错误回传修正；对结果做合理性检查；关键结论附带执行的 SQL 便于核对。

62. 何时需要人工确认

- **不可逆或高影响操作**：删除数据、资金交易、生产环境变更、对外发送消息/邮件；
- **权限边界**：首次访问敏感资源、提升权限；
- **不确定性高**：用户意图模糊、多个合理方案需取舍、模型置信度低；
- **策略与合规要求**：涉及隐私、法律、医疗等领域；
- 设计要点：确认请求要**说明将执行的具体动作、影响范围与理由**；支持批准/拒绝/修改；超时默认不执行；在 LangGraph 中可用 `interrupt` 暂停图执行等待人类输入后恢复。
- 平衡：低风险的读操作自动执行，避免频繁打扰用户导致「确认疲劳」。

十、Agent Harness 与工程化

63. Agent Harness 是什么

- **Harness**：围绕大模型的一套**运行支撑系统**——模型负责理解与决策，工具负责具体操作，Harness 把二者组织成**可持续运行、可控的任务过程**。
- 核心职责：
  - **执行循环**：准备输入 → 调用模型 → 解析输出 → 校验并执行工具 → 将真实结果纳入下一轮；
  - **上下文管理**：决定每轮模型看见什么（裁剪、压缩、按需加载）；
  - **状态与进度**：记录已执行操作、产物和验证结果，支持中断恢复；
  - **权限与安全**：工具权限、沙箱、人工确认；
  - **预算与终止**：步数、时间、token 预算，以及**基于真实证据的完成判断**；
  - **可观测性**：日志、Trace、指标。
- 模型提出的工具调用只是「请求」，**真正执行与结果回传由 Harness 完成**；只给模型一个工具列表不等于有了完整 Harness。

64. 设计完整的 Agent Harness

- **模型层**：多模型适配与路由、重试与 fallback、流式输出、结构化输出校验。
- **工具层**：工具注册与描述、参数 Schema 校验、超时与重试、并行执行、结果裁剪、权限分级。
- **上下文层**：系统提示组装、记忆与 RAG 注入、历史压缩、Prompt Caching 友好的结构。
- **状态层**：会话与任务状态持久化、检查点、幂等写操作、恢复时核对外部状态。
- **控制层**：最大步数/预算、循环检测、人工确认节点、取消与中断。
- **安全层**：沙箱、最小权限、注入防护、审计。
- **观测与评估层**：Trace、成本统计、离线评测与线上回归。

65. Agent 自进化机制

- 目标：Agent 随使用不断变好，而不是每次都从零开始。
- 实现方式：
  - **经验记忆**：从成功/失败轨迹中提炼经验与规则，写入长期记忆或程序性记忆，后续检索复用；
  - **反思沉淀**：Reflexion 式反思结果持久化；
  - **技能库**：把验证有效的操作流程沉淀为可复用的 Skill / 工具（如 Voyager 在 Minecraft 中构建技能库）；
  - **Prompt 自动优化**：基于评测反馈迭代提示（如 DSPy 类方法）；
  - **数据飞轮**：收集线上轨迹与用户反馈，用于微调或偏好训练。
- 风险控制：新经验需经验证再生效、可回滚、防止记忆被恶意污染。

66. 用 Go 开发 Agent 的工程要点

- **流式输出**：使用 SSE（`text/event-stream`），配合 `http.Flusher` 逐块下发；处理客户端断开（`r.Context().Done()`）及时停止生成以节省成本。
- **并发调用工具**：无依赖的工具调用用 goroutine + `errgroup` 并行执行，`SetLimit` 控制并发度。
- **超时与取消**：所有模型调用、工具调用传递 `context.Context`，`WithTimeout` 控制单步和整体超时，用户取消时级联取消。
- **重试与限流**：对 429/5xx 做指数退避重试；`golang.org/x/time/rate` 做客户端限流；多 Key/多模型做负载均衡与 fallback。
- **结构化输出**：用 struct + JSON Schema 描述工具参数，`encoding/json` 严格解析并校验，失败信息回传模型。
- **生态**：字节跳动 CloudWeGo 开源的 **Eino**（Go 的 LLM 应用框架）、langchaingo、官方/社区 MCP Go SDK。

十一、开发框架

67. 常见 Agent 开发框架

- **LangChain**：最流行的 LLM 应用开发框架，提供模型接口、Prompt、工具、检索、Agent 等组件与大量集成。
- **LangGraph**：LangChain 团队推出的**基于图的有状态 Agent 编排框架**。
- **LlamaIndex**：侧重**数据接入、索引与 RAG**。
- **AutoGen**（微软）：多 Agent 对话式协作；**CrewAI**：基于角色和任务的多 Agent 协作；**MetaGPT**：用 SOP 模拟软件团队。
- **OpenAI Agents SDK**：轻量的 Agent、工具、handoff、guardrail 原语。

- **Java 生态**：LangChain4j、Spring AI；**Go 生态**：Eino、langchaingo。
- **低代码平台**：Dify、Coze 等，可视化编排工作流与知识库。

68. LangChain 核心概念与架构

- **Chain 的理解**：把多个组件（Prompt 模板 → 模型 → 输出解析器 → 检索器等）**按顺序组合成一个可复用的处理流程**，前一步输出作为后一步输入。设计理念是**组件化 + 可组合**。
- **LCEL（LangChain 表达式语言）**：用 `|` 管道符组合 **Runnable**，如 `prompt | model | parser`；所有组件实现统一的 Runnable 接口（`invoke`、`batch`、`stream` 及异步版本），自动支持流式、批处理、并行（`RunnableParallel`）和重试/回退。
- **包结构**：`langchain-core`（核心抽象与 LCEL）、`langchain`（链、Agent 等）、`langchain-community`（社区集成）、partner 包（如 `langchain-openai`）。
- **核心组件**：Models（Chat Model、Embedding）、Prompts、Output Parsers / 结构化输出、Document Loaders、Text Splitters、Vector Stores、Retrievers、Tools、Agents、Memory、Callbacks（可观测）。

69. 用 LangChain 构建 Agent 与注册工具

- 核心步骤：
  1. 选择支持工具调用的 **Chat Model**；
  2. **定义工具**：用 `@tool` 装饰器修饰函数（函数名、docstring、类型注解自动生成工具名、描述与参数 Schema），或用 `StructuredTool` / Pydantic 模型定义参数；
  3. 编写**系统提示**；
  4. 创建 Agent（新版本使用 `create_agent`，旧版本为 `create_react_agent` / `AgentExecutor`）；
  5. 配置**记忆/检查点**、最大迭代次数等，调用并处理流式输出。
- 工具描述要清楚说明用途和参数含义，模型主要依据描述选择工具。

70. LangChain 的记忆实现

- **短期记忆**：会话内消息历史。新版本基于 LangGraph 的 **checkpointer**，以 `thread_id` 区分会话，自动持久化每一步状态（内存、SQLite、Postgres、Redis 等后端）；旧版本有 `ConversationBufferMemory`、`ConversationSummaryMemory` 等。
  - 消息过长时可裁剪（trim）或摘要。
- **长期记忆**：跨会话存储，LangGraph 提供 **Store**（按命名空间存取 JSON 文档并支持语义检索），也可自行接入向量库保存用户偏好与历史事实。

71. LangChain 与 LlamaIndex、LangChain4j

- **LangChain vs LlamaIndex**：LangChain 是**通用编排框架**，组件全面、适合复杂 Agent 与工作流；LlamaIndex 专注**数据层与 RAG**，在文档解析、索引结构、查询引擎方面更深入、开箱即用。可结合使用。

- **LangChain4j**：Java 生态的 LLM 框架，解决 Java 开发者接入大模型的问题：统一多模型 API、**AI Services**（用 Java 接口 + 注解声明式定义 AI 能力）、工具调用（`@Tool` 注解方法）、RAG 组件、记忆、与 Spring Boot / Quarkus 集成。适合**已有 Java 企业系统中集成 AI 能力**。

72. LangChain 与 LangGraph 的区别

- **LangChain**：以**链（线性/DAG）**为主的组件库，适合流程相对固定的应用；早期 AgentExecutor 循环逻辑封装较深，难以定制。
- **LangGraph**：把 Agent 建模为**有状态图**：
  - **State**：在节点之间共享的状态（TypedDict / Pydantic），通过 reducer 合并更新；
  - **Node**：执行单元（调用模型、调用工具或任意函数）；
  - **Edge / 条件边**：决定下一步走向，**原生支持循环**；
  - **Checkpointer**：每步持久化状态，支持**中断恢复、时间回溯（time travel）**；
  - **Human-in-the-loop**：`interrupt` 暂停等待人工输入；
  - 流式输出（token 级、节点级）、子图、多 Agent（Supervisor、Swarm）。
- **核心优势与适配场景**：需要**循环、分支、长时间运行、可恢复、人工审批、多 Agent 协作**的复杂 Agent，以及对执行流程有精细控制要求的生产场景。
- 关系：LangGraph 更底层、可控；LangChain 提供组件与高层封装，两者常配合使用。

73. LangChain 大版本演进

- **v0.1（2024 年初）**：拆分为 `langchain-core`、`langchain-community`、`langchain` 及 partner 包，稳定核心抽象，LCEL 成为主要组合方式。
- **v0.2 / v0.3（2024）**：进一步减少对 community 的依赖，v0.3 全面迁移到 **Pydantic 2**；推荐用 LangGraph 构建 Agent，旧 AgentExecutor、旧式 Memory 逐步弃用。
- **v1.0（2025 年 10 月，与 LangGraph 1.0 同期）**：以 `create_agent` 作为标准 Agent 构建方式，**构建在 LangGraph 运行时之上**；引入 **Middleware 中间件**机制（在模型调用前后插入摘要、人工审批、PII 处理等逻辑）；标准化消息内容块；精简包结构，旧版链与工具迁移到 `langchain-classic` 包。
- 面试提示：说明自己使用的版本，并以官方迁移文档为准。

74. Deep Research 的实现逻辑

- **定义**：针对复杂问题自主进行**多轮搜索、阅读、分析与综合**，最终生成带引用的长篇研究报告的 Agent 应用。
- **典型流程**：
  1. **需求澄清**：必要时向用户追问研究范围；
  2. **规划**：拆解为多个研究子问题/方向；

3. **迭代检索**：搜索 → 阅读网页/文档 → 提取要点 → 根据新发现**生成新查询**，直到信息充分或达到预算；
4. **反思与查漏**：检查是否覆盖所有子问题、是否有矛盾信息；
5. **综合写作**：按大纲生成报告并**标注引用来源**。

- **架构**：常用 **Orchestrator + 多个并行研究子 Agent**（各自独立上下文搜索不同方向，返回压缩后的发现），由主 Agent 汇总写作。
- **适用场景**：行业/竞品调研、技术选型、文献综述、尽职调查等**开放式、信息量大、需要多源整合**的任务；不适合有确定答案的简单查询（成本高、耗时长）。